

/se

PRÁCTICA 1

P1-I1. ubuntu server

Aquí nos piden crear el ubuntu server con tres volúmenes lógicos, para ello lo que hacemos primeramente es meter dos discos ya que también nos piden hacer un raid1 por lo que estamos obligados a tener dos discos.

Primeramente creamos dos particiones en cada uno de los discos para hacer la partición de booteo

una vez tenemos las dos particiones lo que hacemos es crear el primer raid1 con estos dos para que si falla un disco para el booteo que podamos bootear aun así el sistema.

ahora ya hacemos una partición en cada uno de los discos con el resto del espacio

cuando ya tenemos las particiones de los dos discos, lo que hacemos es crear un raid1 con estas dos particiones

una vez que ya tenemos el raid creado ya podemos crear el grupo de volúmenes con este raid ahora ya hacemos cada volumen lógico, tener en cuenta que raíz deberá tener todo el resto del espacio sobrante de /home y swap

P1-I2

Nos piden agrandar /var

Que deberíamos hacer?

Crear nuevo disco

Particionarlo con fdisk

Después de crearlo necesitamos un volumen físico, para ello usamos `pv create /dev/sdb`

ya tenemos el volumen físico

ahora el grupo de volúmenes, podemos reutilizar el que ya tenemos con `vgextend rl_vbox /dev/sdb1`

cuando ya tenemos el grupo de volúmenes asignado nos falta el volumen lógico, como nos dice que hagamos uno exclusivo para esto, creamos uno nuevo, esto se haría con `lvcreate`

tenemos que tener en cuenta que `lvcreate` necesita parámetros específicos

`lvcreate -L(tamaño) 4G -n(nombre) newvar rl_vbox`

ahora que tenemos todo esto necesitamos montarle primero el tipo ext4 y montarlo

`sudo mkfs -t (tipo) ext4 /dev/rl_vbox/newvar`

ahora solo nos queda el montaje, como nos dicen que /var se queda sin espacio entonces

tenemos que montarlo sobre este archivo, como no queremos perder todo lo que tenemos

primero creamos un nuevo directorio donde copiamos todo lo de /var y allí es donde montamos

primeramente todo ya que no podemos montarlo sobre /var directamente, para ello

`sudo mkdir /mnt/newvar`

`mount /dev/rl_vbox/newvar /mnt/newvar`

`mount`

ahora copiamos todo de /var en /newvar

como tenemos que tener en cuenta que hay a lo mejor gente que está metiendo cosas en /var en el momento en el que queremos copiar todo nos metemos en modo aislamiento

`systemctl isolate rescue`

ahora sí que podemos copiar

`cp -a /var/. /mnt/newvar`

DATO CURIOSO: nosotros primero montamos en /mnt/newvar. ya que si primero copiamos y luego montamos perdemos todo, entonces tenemos primero que montarlo y luego ya copiar, así no perdemos ni lo de /var ni lo que íbamos a copiar en /newvar

ahora que ya tenemos todo copiado ya podemos en /etc/fstab montar lo que ya está montado sobre /mnt/newvar sobre /var así accedemos a tener más espacio

modificamos fstab y metemos la siguiente línea

`/dev/rl_vbox/newvar /var ext4 defaults 0 0`

ahora si le damos a `mount -a` forzamos el montaje sobre /var

finalmente como ya tenemos lo nuevo montado sobre /var ya podemos desmontar /newvar

`umount /mnt/newvar`

P1-l3 rocky

como nos pide un raid1 y nosotros ya tenemos el disco1 ocupado entonces si creamos el raid1 usando ese disco, nos ahorraríamos crear un nuevo disco sin embargo, perderíamos espacio ya que como en el raid1 se copia lo de uno en otro entonces si en uno están ocupadas 4gigas en el otro igual así que nos renta más crear dos discos nuevos y proceder con el raid1 desde el principio

entonces primeramente metemos dos discos nuevos y hacemos la partición en los dos para poder usarlos desde ya

una vez que tengamos las particiones ya hechas creamos el raid 1

`mdadm --create /dev/md0 -level=1 (ya que queremos RAID1) --raid-devices=2 /dev/sdb1 /dev/sdc1`

cuando ya tenemos el raid1 creado ya pasamos al volumen físico

`pvcreeate /dev/md0`

luego el grupo de volúmenes, podríamos reutilizar `rl_vbox` pero esta vez no renta ya que perderíamos el tracking de donde creamos las cosas entonces creamos un nuevo grupo de volúmenes

`vgcreate vg_raid1 /dev/md0`

después del grupo de volúmenes pasamos a los volúmenes lógicos

`lvcreate -L 1G -n newvar vg_raid1`

ahora tenemos que cifrar el VL ya que nos dice la cliente que quiere un VLCifrado

instalamos `cryptsetup`

usamos LUKS on LVM

`cryptsetup luksFormat /dev/mapper/vg_raid1-newvar`

ahora tenemos que abrir el cifrado ya que ya tenemos el cifrado creado pero nos falta abrirlo sobre un archivo

`cryptsetup luksOpen /dev/mapper/vg_raid1-newvar vg_raid1-newvar_crypt` (le asignamos el nuevo nombre a dicha entidad abierta)

ahora que ya tenemos todo ya repetimos lo de la práctica anterior que sería montar el tipo `ext4` y montarlo sobre un nuevo archivo copiar y luego montar sobre `/var` de nuevo

establezco el tipo de archivo con `mkfs -t ext4 ....`

creo directorio, monto sobre este directorio, luego copio todo lo de `/var` en él y finalmente monto todo esto sobre `/var`

lo último es desmontar ya lo que queda, desmontar el archivo que acabamos de crear

finalmente para acabar nos falta el último paso

- `blkid | grep crypto >> /etc/crypttab`
- editamos con `vi` dicho archivo → `vi /etc/crypttab`

PRÁCTICA 2

P2-L1. SSH

sabemos que `sshd` es la confi del servidor y `ssh` del cliente

por otro lado sabemos que el firewall de ubuntu no viene iniciado mientras que el de rocky si para instalar `ssh` en ubuntu usamos `sudo apt install openssh-server` y lo iniciamos con `systemctl`, vemos que por defecto viene el puerto 22

para modificar el puerto entramos en `/etc/ssh/sshd_config`

modificamos el puerto y el `permit root login` a no y descomentamos todo

ahora si reiniciamos `ssh` veremos que ya podemos acceder con este puerto 22022

en rocky ya viene `ssh` instalado para servidor y cliente

para evitar meter contraseñas creamos una clave pública en rocky y se la pasamos a ubuntu `ssh-keygen`

así ya tenemos la clave pública, ahora se la tenemos que pasar a ubuntu con

`ssh-copy-id -p 22022 mehdii@192.168.56.105`

hacemos lo mismo con el peuerot en rocky sinn. embargo leemos y vemos que ademas de todo lo que hicimos en ubuntu debemos ejecutar un comando que viene en el archiv /etc/ssh/sshd_config

para ello usaremos semanage sin embargo para instalarlo descargamos polycycoreutils-.....

al intentar acceder no nos deja tampoco y eso es por lo que dije del firewall, el firewall en rocky viene iniciado ya de por si por lo que lo que hacemos es

```
sudo firewall-cmd --add-port=22022/tcp --permanent
```

ahora habilitamos el firewall en ubuntu con sudo ufw enable

ahora sudo ufw allow 22022

P2-L2. copias y control de versiones

Para copias tenemos diferentes metodos: dd(que permite hacer copias completas de disco), cp (copia), cpio(en desuso) y rsync(herramienta útil que permite hacer copias de una maquina a otra maquina remota usando ssh)

EJ: rsync -re(copia en remoto) "ssh 22022"(para indicar que no se usa el puerto 22 sino que el puerto esta cambiado) --progress(para mostrar el progreso) /home/mehdii/dir1/.

mehdii@192.168.56.110:/home/mehdii/dirCopia

Ahora ya pasamos a git, que nos ayuda a saber todos los cambios que se van haciendo sobre nuestros archivos y quien los ha realizado. Lo hacemos todo en rocky

Primero antes de todo debemos meter nuestro correo y nombre para saber quien hace los cambios

```
git config --global user.email "iaboutenmehdi@correo.ugr.es"
```

```
git config --global user.name "ERMEDI"
```

Vamos a poner todos los comandos que existen y cuales son sus funcionalidades ya que es lo unico que podrian preguntar en el examen:

- git init (para inicializar el servicio git)
- git add file.txt (para añadirlo, esto es lo primero que tenemos que hacer siempre sino no podemos hacer ningun commit ni nada, o sino la combinacion de commit y add que es la que viene en la siguiente linea)
- git commit -a -m "Cambio a puerto 22022"
- git log --graph --all --decorate (para ver el arbol bien y poder identificar las diferentes ramas)
- git checkout "primeros numeros del identificador de la rama". (esto nos sirve para movernos a otro commit, es decir, mover el head)
- git branch (para ver todas las ramas que tenemos)
- git checkout -b config-selinux. (creamos rama nueva y nos vamos a ella)
- git merge config-selinux config-puertos. (sirve para combinar dos ramas, esto hace que se combinen tambien los archivos. Ahora si modificamos el archivo como queramos y hacemos el commit de la combinacion)

```
# semanage port -a -t ssh_port_t -p tcp #PORTNUMBER
<<<<<< HEAD
# YA LO HE CONFIGURADO
Port 22022
=====
#
Port 220
>>>>>> config-puertos
```

P2-L3. pila lamp, fail2ban, screen **fail2ban**

Vamos a empezar con fail2ban, que nos sirve para banear a usuarios sospechosos, es decir, cuando prueba muchas veces contraseñas(limite de 5), etc
Para ello primero debemos instalarlo con `dnf install epel-release -y` y `dnf install fail2ban -y`
Cuando ya lo tenemos activo y todo ya podemos entrar al status con `fail2ban-client status` y vemos las jails que tenemos configuradas
Ahora si entramos a `/etc/fail2ban/jail.conf` podemos crear las nuevas jails
Como queremos establecer esta proteccion para ssh vamos al apartado donde pone SSH SERVERS y modificamos el archivo metiendo `[sshd] enabled=true`, y cambiamos el puerto a 22022 ya que nosotros no estamos usando el puerto por defecto de ssh
Una vez que ya lo tenemos ya haciendo el status vemos que tenemos a jail de ssh activa, esto hace que si alguien intenta conectarse por ssh y falla la contraseña 5 veces se le banea para quitar el ban usamos `fail2ban-client set sshd unbanip 192.168.56.1`

screen

`dnf install screen -y`
nos ayuda a no perder todo lo que estabamos haciendo anteriormente
si hacemos screen se nos abre una pestaña vacia en la terminal
cuando hacemos `ctrl+a+d` volvemos a la pantalla anterior
`screen -list`
`screen -r` (numero de sesion) para conectarnos a una sesion anterior

pila lamp (linux, apache, mysql, php)

en ubuntu lo instalamos en un momento con `tasksel`
como en ubuntu usamos apache2 necesitamos deshabilitar el firewall para el puerto 80 que es el de httpd/apache con `ufw allow 80`

Montaje pila LAMP en rocky:

HTTPD: `dnf install httpd -y` y lo siguiente seria activarlo. Por último debemos desactivar el firewall para el puerto 80 con `firewall-cmd --add-port=80/tcp`. Para comprobar que funciona correctamente ponemos la ip de rocky y de ubuntu en google y deberia salirnos una pagina de inicio de apache2(ubuntu)/httpd(rocky).

MYSQL (mariadb en rocky): `dnf install mariadb -y` // `dnf install mariadb-server -y` y ya lo activamos y reiniciamos. Ahora ya podemos comprobar su funcionamiento con `mysql -u root -p`

PHP: `dnf install php -y` es un interprete no un servicio entonces `systemctl` va a dar error siempre. Para meter nuestro php nos vamos a `/var/www/html` y creamos nuestro php. Metemos todo lo necesario, AVISO(no podemos usar root asi que tenemos que crear nuevo usuario y darle privilegios, todo esto con mysql. Ademas de eso tenemos que crear la base de datos que vamos a usar). Una vez que tenemos nuestro php creado lo metemos en `/etc/httpd/httpd.conf` donde pone `index.html` que es lo que sale normalmente ponemos al lado el `miscript.php`.
Primero para solucionar el error que nos deberia salir es descargar `dnf install php-mysqlnd -y` ya que la funcion `mysqli_connect`. Finalmente ya lo tenemos funcionando si ya hemos creado el nuevo usuario y le hemos dado permisos y modificado el php.
Ahora si hacemos `php miscript.php` ya funciona. Lo unico que nos falta es poder visualizarlo que se haría con el `setsebool httpd_can_network_connect_db on` ya que por defecto viene en OFF

AHORA SÍ YA TENDRÍAMOS TODA LA PILA LAMP FUNCIONANDO.

PRÁCTICA 3 zabbix y ansible

zabbix

empezamos con zabbix que nos permitira pues toda la monitorizacion del sistema, nos permite ver grafos del funcionamiento del sistema
nosotros ademas del host que sera ubuntu donde hemos creado el servidor, debemos crear otro perfil para rocky donde metemos la ip de rocky

en cuanto a los templates, deberemos meter HTTP SERVICE y SSH service en ambos ya que es lo que queremos comprobar en ambos casos. metemos estos templates en ambos servidores ademas de esto deberemos de tener en cuenta de nuevo que tenemos el puerto cambiado por lo que debemos modificar el template SSH SERVICE para que no intente acceder por el puerto 22 y lo haga por el 22022 que es por donde escuchan ambos servidores las peticiones de ssh para ello lo que haremos será entrar en el template de ssh service y poner en el apartado key [ssh,,22022] configurandolo así para que escuche de este puerto con esto ya lo deberiamos de tener todo configurado, con entrar al grafo de ssh service podemos ya mirar los cambios donde si desactivamos el ssh por ejemplo y lo volvemos a activar deberiamos ver como baja y sube la representacion en la grafica

ansible

Playbooks: secuencia de acciones a tomar, es como un script/receta

sudo apt install ansible -y

ansible all --list-hosts para ver los hosts configurados

los configuramos de esta forma -> [nombre] 192.168.56.11:22022

como vamos a hacer uso de ssh para usar ansible entonces necesitamos configurar claves

publicas para todos los servidores que vayamos a usar con el ansible (ssh-keygen. ssh-copy-id

192.168.56.105 -p 22022)

“ansible all/rocky/ubuntu/ambos(aquí es donde metemos a q host le vamos a realizar la acción o a todos) -m command -a “mkdir /home/mehdii/ansiblePrueba“

“ansible all -m ping”

ahora ya sabemos trabajar con ejecuciones simples con ansible, así que pasamos a los playbooks que son .yaml

```

- name: comprobarRocky
  hosts: rocky
  become: true
  become_user: root
  tasks:
    - name: comprobarVersionHTTPD
      ansible.builtin.dnf:
        name: httpd
        state: latest

    - name: comprobarHTTPDOperativo
      ansible.builtin.systemd:
        name: httpd
        state: started

- name: comprobarUbuntu
  hosts: ubuntu
  become: true
  become_user: root
  tasks:
    - name: comprobarVersionApache
      ansible.builtin.apt:
        name: apache2
        state: latest

    - name: comprobarApacheOperativo
      ansible.builtin.systemd:
        name: apache2
        state: started

```

Esta es la estructura de este tipo de archivos.

PUNTOS A DESTACAR:

- lo hacemos desde root porque vamos a descargar y actualizar cosas que eso solo se puede realizar con el root , para la contraseña meteremos el flag -K que nos permite meter la contraseña
- debemos tener en /etc/ansible/hosts las ips de cada servidor cada una sola ya que como uno usa dnf otro apt y uno apache2 y le otro httpd entonces no podemos hacer lo mismo pa los dos
- state latest lo que hace es comprobar si esta en la ultima version y en caso de no estarlo lo que hace es actualizarlo
- state started comprueba si el servicio esta inicializado
- ansible.builtin.apt o ansible.builtin.dnf

ansible-playbook -i (para indicar donde esta el archivo de los hosts) /etc/ansible/hosts
comprobacion.yaml -K (para meter la contra)